

Local-first Coordination via Friend-to-Friend Choreographies^{*}

Vincenzo Ciancia¹

Consiglio Nazionale delle Ricerche, Istituto di Scienza e Tecnologie dell'Informazione "A. Faedo"

Roberto Guanciale

KTH Royal Institute of Technology

Daniele Strollo

Alumnus IMT, Scuola Alti Studi Lucca

Emilio Tuosto^{*}

GSSI, L'Aquila, Italy

Dragiša Žunić

The Institute for Artificial Intelligence Research and Development of Serbia, Novi Sad, Serbia

Abstract

We lay out a new choreographic model that, departing from point-to-point interactions, aims to harness local-first coordination of distributed applications. We dub our model *friend-to-friend* choreographies since its main application domain are decentralised applications that guarantee confidentiality in a *fully decentralised* setting. A key principle of local-first coordination is indeed that the control on computational resources is not given to third-parties. This yields a notion of projection different than those usually found in the literature since our projection operation aims to identify minimal sets of agents that should be aware of an interaction and so compute the cryptographic keys necessary to ensure confidentiality.

Friend-to-friend choreographies have a straightforward implementation in Nearbybytes, a platform which provides a communication infrastructure and friend-to-friend decentralised storage.

Keywords: Choreography, Local-first coordination, Friend-to-Friend architectures

1. Introduction

- 2 Distributed applications are increasingly moving away from cloud-centric architectures towards loosely
coupled systems, where users control their data while still benefiting from collaboration and using suitable
4 synchronisation mechanisms. This shift is motivated by the desire to reduce dependencies on trusted third

^{*}We sincerely thank the anonymous reviewers whose comments, insights, and suggestions greatly helped us to improve the content and the presentation of the paper.

^{*}Corresponding author

Email addresses: vincenzo.ciancia@isti.cnr.it (Vincenzo Ciancia¹), robertog@kth.se (Roberto Guanciale²), daniele.strollo@gmail.com (Daniele Strollo), emilio.tuosto@gssi.it (Emilio Tuosto³), dragisa.zunic@ivi.ac.rs (Dragiša Žunić⁴)

¹Author supported by the CNR project "Formal Methods in Software Engineering 2.0", CUP B53C24000720005

parties. Recently, *local-first coordination* [1] advocates local computation and disconnected operations in order to let applications control their computational and data resources [2, 3, 4, 5]. This enables applications to continue to operate under intermittent connectivity by distributing the computational state among peers rather than delegating it to third-party servers.

Our work is motivated by Nearbybytes [6], a decentralised infrastructure for local-first communication and storage that organises interactions through cryptographically protected shared logs. Following the suggestion in [7, 8] to ground peer-to-peer coordination on trust emerging from “social relationships”, and friendship in particular, Nearbybytes provides mechanisms for secure dissemination of events. However, designing distributed applications in such settings is challenging. Developers must simultaneously reason about application logic, message distribution, and confidentiality constraints.

Choreographies provide a promising abstraction to tackle this complexity. Rather than describing the behaviour of each component independently, choreographies specify interactions from a global perspective and derive local behaviours through projection. On the one hand, existing choreographic models, however, have been primarily developed for message-passing systems where interactions are fundamentally point-to-point. As a consequence, they are not naturally suited for coordination settings where interactions are one-to-many and where visibility of events may differ from visibility of their content. On the other hand, approaches such as those in [2, 4] are designed for *closed* settings where confidentiality is not an issue.

To fill this gap, we introduce *friend-to-friend choreographies* (F2F-choreographies), a new coordination model featuring local-first coordination for confidential interactions. The gist of our idea is to combine principles of local-first coordination with choreographic approaches. The global view of F2F-choreographies models interactions as events intentionally shared among subsets of agents, abstracting away from the “cryptographic” infrastructure necessary to ensure the confidentiality of the communications. This allows a separation of concerns so that designers can focus on the intended information flow of the protocol. Confidentiality aspects are then taken care of by a *projection operation* designed to identify which agents must be aware of an interaction and which ones can remain oblivious to it. Also, the local models of F2F-choreographies are formalised as symbolic automata interacting through *hubs*, that can be thought of as structured decentralised data spaces. Such automata specify the behaviour of agents that can be realised through the API of Nearbybytes.

More specifically, this paper makes the following contributions:

- we introduce F2F-choreographies (cf. Section 3), a choreographic model inspired by g-choreographies (see Section 2.1 for an overview) tailored to local-first and decentralised infrastructures, whose present local view is formalised as a class of symbolic automata (cf. Section 4) with an abstract execution model inspired by Nearbybytes (cf. Section 2.2);
- we define a projection algorithm (cf. Section 5) that derives local automata and computes the minimal dissemination structure necessary to execute a choreography;
- we discuss correctness (cf. Section 6) and illustrate the approach through a realistic workflow example (cf. Section 7);
- we discuss how Nearbybytes can use our automata to execute the choreography (cf. Section 8).

This work is an initial step towards an interesting model for local-first coordination ensuring confidentiality in a fully decentralised setting. We discuss limitations, future directions, and related work in Section 9.

Grazie, Giangli!

The title of this paper hints to the main reasons for writing it: we wanted to demonstrate our immense gratitude and our feelings to Giangli.

On the one hand, we meant to give a tribute to Giangli’s scientific activity by blending together topics that we worked on together under his mentoring; we all have fond memories of the discussions on network-awareness, security, models of distributed computations, paradigms for distributed programming that we

had during our studies in Pisa that shaped our professional growth. Giangi shared with us his vast knowledge and views on all these aspects of our discipline and transmitted to us the passion for our work. In particular, we followed the main inspiring principle that there should be continuous interplay among theory, formal modelling, rigorous analysis, and programming primitives. This paper is intentionally positioned at the intersection of several ideas that have accompanied us since our PhD years with Giangi. Local-first, friend-to-friend coordination is a relatively recent area which we were able to explore following Giangi's style and, ultimately, largely reusing what he taught us. A recurring theme we've been exploring with Giangi is the tension between a global, organised understanding of collective behaviour, and its effective decentralised implementation. Distributed systems are often easier to specify from a global viewpoint, where dependencies and interactions are explicit, yet they must ultimately execute through autonomous agents with only partial knowledge of the system. Reconciling these two perspectives has been a central challenge in much of our work with Giangi and naturally resurfaces in the context of local-first, friend-to-friend coordination.

The technical choices presented in this paper are also guided by these principles. In particular, we use multicast interactions and groups of agents, rather than individual point-to-point exchanges, at the centre of the coordination model, reflecting the communication patterns naturally arising in decentralised infrastructures.

On the other hand, this work has a meaning beyond its technical contribution, a twofold meta-meaning one could say. Firstly, 'networking' is one of the great lessons we learned from Giangi: collaborating with Giangi meant for us being part of an international research network within which we still operate. Secondly, 'friend-to-friend' reflects our own scientific journey with Giangi, which grew from a collaboration among advisor and students into a lasting friendship. Giangi's attitude has shaped the way we approach our work: it's not just about papers, it's about friendship!

2. Background

2.1. Pomsets and choreographies

We review the concurrency model used to formalise the semantics of our framework. Most of the material is adapted from [9]; for a complete treatment the reader is referred to [10, 11, 12, 13].

Definition 1. *Given a set of events $\mathcal{E}$, a labelled partially-ordered set (lposet) is a triple $(\mathcal{E}, \hookrightarrow, \lambda)$, where*

- $\hookrightarrow \subseteq \mathcal{E} \times \mathcal{E}$ is a reflexive, anti-symmetric, and transitive relation on $\mathcal{E}$ called happens-before relation,
- and $\lambda : \mathcal{E} \rightarrow \mathcal{L}$ is a labelling function mapping events in $\mathcal{E}$ to labels in $\mathcal{L}$.

Two lposets $(\mathcal{E}, \hookrightarrow, \lambda)$ and $(\mathcal{E}', \hookrightarrow', \lambda')$ are isomorphic if there is a bijection $\phi : \mathcal{E} \rightarrow \mathcal{E}'$ such that $e \hookrightarrow e' \iff \phi(e) \hookrightarrow' \phi(e')$ and $\lambda = \lambda' \circ \phi$.

Intuitively, λ labels observable events of the system and $\hookrightarrow$ is a partial order representing causality. Since λ is not required to be injective, there might be different events labelled with the same action. When events $e \neq e'$ both occur and $e \hookrightarrow e'$ then e is a cause of e' or, equivalently, e' causally depends on e . Indeed, events' identities are immaterial, hence Definition 1 identifies isomorphic lposets in order to abstract away from the names of events in $\mathcal{E}$.

Definition 2 ([14, 12]). *Given an lposet $(\mathcal{E}, \hookrightarrow, \lambda)$, a pomset is a class $[\mathcal{E}, \hookrightarrow, \lambda]$ of lposets isomorphic to $(\mathcal{E}, \hookrightarrow, \lambda)$. We let $r, r', \dots$ range over pomsets and $R, R', \dots$ range over sets of pomsets.*

We lift the order relation $\hookrightarrow$ to set of events by stating that $E \hookrightarrow E'$ holds when $e \hookrightarrow e'$ for all $e \in E$ and $e' \in E'$. Being isomorphism classes, pomsets are non empty; we will refer to the representative element of a pomset r as $(\mathcal{E}_r, \hookrightarrow_r, \lambda_r)$ and we may write $e \in r$ for $e \in \mathcal{E}_r$.

Definition 3. *Fix a pomset r . An event $e \in r$ is an immediate cause of event e' (or equivalently e' is an immediate consequence of e) if $e \neq e'$, $e \hookrightarrow_r e'$, and for all $e'' \in \mathcal{E}_r$ such that $e \hookrightarrow_r e'' \hookrightarrow_r e'$ either $e = e''$ or $e' = e''$. We let $\hat{e}$ denote the set of immediate causes of e .*

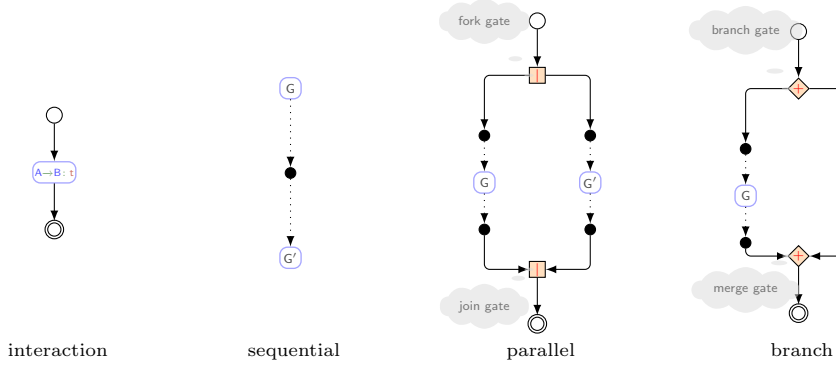


Figure 1: Diagrammatic Representation of the Syntax of Global Choreographies

We survey our model of global views of choreographies aiming to describe interactions among distributed *agents* interacting with each other. For simplicity, we consider only the finitary fragment introduced in [15].

Definition 4 (cf. [15]). A (finitary) global choreography (*g-choreography for short*) is a term G derivable from the following grammar:

$$G ::= A \rightarrow B : t \mid G; G' \mid G \mid G' \mid G + G'$$

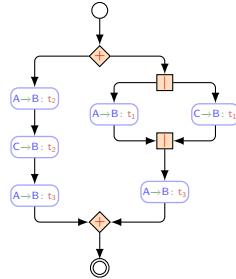
where A and B range over agents' identities and t ranges over (types of) messages.

The basic building blocks of this syntax are two-party interactions, written $A \rightarrow B : t$, that specify that an agent A sends a message of type t to another agent B , which in turn receives it. G-choreographies can be composed in sequence, in parallel, or in non-deterministic choice.

A reason for choosing g-choreographies is that they have a diagrammatic counterpart. Indeed, we can constructively obtain the diagrammatic representation from the syntax of g-choreographies; here we give an intuitive account of how this transformation works using Fig. 1; the interested reader is referred to [15] for the formal definition. The idea is to depict G as a graph with an “interface” made of a rooted *source* node $\circ$ and a single *sink* node $\odot$. The inner nodes of the graph represent interaction, branch, or fork points. Interaction nodes are those labelled with the interaction they represent while *gate* nodes $\square$ and $\diamond$ respectively identify fork and branch as well as their corresponding “closing” points.

In Fig. 1, a boxed G represents the diagrammatic representation of G and a dotted edge from $\bullet$ to a boxed G represents the fact that we connect the gate entering in $\bullet$ to the source node of the boxed G while removing the source node of G . Likewise, a dotted edge from a boxed G to $\bullet$ means that the gate arriving in the sink node of G has to be connected to the gate after $\bullet$ and that the sink node of G is removed.

Example 5. (Adapted from [15]) Let $G_1 = A \rightarrow B : t_2; C \rightarrow B : t_2; A \rightarrow B : t_3$ and $G_2 = (A \rightarrow B : t_1 \mid C \rightarrow B : t_1); A \rightarrow B : t_3$. The graphical representation of the choreography $G = G_1 + G_2$ is



The diagrammatic representation of the sequential composition of the interactions in G_1 is just the “chaining” of the diagrams of each interaction; likewise, the representation of G_2 is the chaining of the representation of the parallel composition in G_1 using the $\square$ and the one of the interaction $A \rightarrow B : t_3$; finally, the choice in G combines such diagrams using the $\diamond$ and “wraps” the result within the $\circ$ and $\odot$ nodes. $\diamond$

The semantics of g-choreographies can be given in terms of pomsets (cf. Definition 2) on their communication events [15, 9]. The pomsets for g-choreographies order communication events, namely events labelled by actions of the form $AB!t$ or of the form $AB?t$. The former represents the sending action of a message t from an agent A to an agent B ; the latter, dually, represents the input executed by an agent B of a message t sent by A . The semantics of a g-choreography G can be defined as a set of partial orders $\llbracket G \rrbracket$, one for each branch of the choices in G . This set can be defined by induction on the syntactic structure of G . The base cases are interactions $A \rightarrow B : t$: we have that $\llbracket A \rightarrow B : t \rrbracket$ is a singleton set containing a pomset where an event labelled with $AB!t$ (dubbed *output event*) precedes the one labelled $AB?t$ (dubbed *input event*). Formally,

$$\llbracket A \rightarrow B : t \rrbracket = \left\{ \left[AB?t \longrightarrow AB!t \right] \right\}$$

For the inductive cases, we have

- $\llbracket G + G' \rrbracket = \llbracket G \rrbracket \cup \llbracket G' \rrbracket$ (namely, the semantics of a choice $G + G'$ is the union of the semantics of the branches G and G');
- the semantics of $G; G'$ merges the pomsets obtained by the Cartesian product of the pomsets of $\llbracket G \rrbracket$ and $\llbracket G' \rrbracket$, adding dependencies that force events of the same agent in G to precede those in G' ;
- the semantics of $G \mid G'$ is obtained by merging the pomsets obtained by the Cartesian product of $\llbracket G \rrbracket$ and $\llbracket G' \rrbracket$, without adding any dependencies;

The reader is referred to [15, 9] for the formal definition.

Example 6. The semantics of the g-choreography G in Example 5 is

$$\llbracket G \rrbracket = \left\{ \left[\begin{array}{ccc} AB!t_2 & \longrightarrow & AB?t_2 \\ \downarrow & & \swarrow \\ AB!t_3 & \longrightarrow & AB?t_3 \end{array} \right] , \left[\begin{array}{ccc} AB!t_1 & \longrightarrow & AB?t_1 \\ \downarrow & & \swarrow \\ AB!t_3 & \longrightarrow & AB?t_3 \end{array} \right] \right\}$$

As observed in [15], this g-choreography cannot be realised without properly coordinating agents; in fact, according to the semantics $\llbracket G \rrbracket$, agents A and C are both supposed to send a message of either type t_1 (left branch) or type t_2 (right branch) to B . This is however immaterial for us. $\diamond$

2.2. Nearbytes: an execution engine for F2F interactions

In a nutshell, Nearbytes is a set of protocols that resemble the standard network stack, but it add to physical wiring communication via the application of symmetric and asymmetric cryptography. Data exchange happens by merging separate encrypted data “hubs”, belonging to different users, through local-first, friend-to-friend coordination. By repeatedly merging hubs, data travels while staying encrypted to preserve confidentiality. Addressing is logical, not physical, and based on public-key cryptography: a public-private key-pair is deterministically generated from a secret; events related to data (e.g., creation, deletion, changes) are signed (the public key is effectively an address of these events) and arranged to form a partially ordered *log*. A strong assumption of Nearbytes is that the semantics of events is deterministic. This enables usage of the *event-sourcing* pattern [16] to replay the log in an application-dependent manner.

The local-first equivalent of “sending”, in our model, is *signing*. A signed message will eventually reach all intended addressees, that is, those who possess the private key or, more precisely, the secret to generate it, by orthogonal, deliberately under-specified means: gossip, USB sticks, email, direct download, a shared folder on a cloud system. Whatever users prefer or is available at a given moment.

The application level is implemented on top of this basic protocol. In this paper, we demonstrate the utility of such a framework by defining the implementation of one specific application level, that of F2F choreographies, on top of it.

2.2.1. Anatomy of a Nearbytes payload

We let K denote a symmetric cryptographic key, K^+ a public key, and K^- the corresponding private key; we write $\{X\}_K$ for the cryptogram obtained by encrypting X with key K using symmetric encryption, $\{X\}_{K^+}$ for the cryptogram obtained by encrypting X with key K^+ using public encryption, and $\{X\}_{K^-}$ for the signature of X using the private key K^- .

As we already mentioned, the core of Nearbytes is a deterministic cryptographic key-derivation function which turns a user-chosen secret into encryption keys. More precisely, given a secret, Nearbytes generates a symmetric key for confidentiality, say K_m , and a public-private key-pair, say K_{hub}^+ and K_{hub}^- , to address a hub and ensure authenticity. Agents' interactions are arranged into *blocks* and *events* involving those keys. Blocks, identified by a digest obtained with a cryptographic hash, yield the content of messages while events “bind” blocks to a list of digests of blocks or other events in order to form a partial order. We call the structure formed by an event, and the set of blocks it references, the *payload*. In this paper, we will use payloads consisting of two encrypted *blocks* and one signed *event*:

```
payload = nb_block_m, nb_block_t, event          where
nb_block_m = {msg}_K_m
nb_block_t = {type}_K_{hub}^+
event = [hash(event_1), ..., hash(event_k)], hash(nb_block_m), hash(nb_block_t)_K_{hub}^-
```

The first block `nb_block_m` carries the actual *content* `msg` of the communication encrypted with the symmetric key K_m ; it is important to remark that the key K_m could in principle be equal to K_{hub} in case all the agents are trusted. The second block `nb_block_t` carries the *message type* `t` encrypted with the public key K_{hub}^+ identifying the hub. Actually, the message and its type could be decoupled; agents could communicate on different hubs (using different key-pairs) and just send the message type over the main coordination hub, maintaining a fully decentralised setup.

The only signed part of the payload is `event`; it contains the list of digests $\text{hash}(\text{event}_i)$ to existing events. Since blocks are content-addressed, an event is small and of fixed size regardless of the size of the content it refers to, and the signature simultaneously (i) authenticates the author, who must know the private key, hence the secret, and (ii) binds the content block and the type block to each other, so that neither can be replaced without invalidating the signature. This is also where the causal dependencies among interactions are recorded, as the references to other events shown above; we will come back to this in Fig. 3.

2.2.2. The architecture of Nearbytes

The reference stack is implemented as a collection of free and open source repositories featuring cryptographic primitives, and append-only event logs to encrypted file volumes, synchronisation, and application wiring [17].

We give a bird's eye view of the stack of Nearbytes whose layered architecture is presented in Fig. 2 in the style of the ISO/OSI reference model; the reader may find it useful as a map of the terminology introduced in the rest of this section. Let us read the stack bottom-up.

- The architecture is parametric with respect to Layer 1. This layer can be any untrusted bearer —a TCP/IP connection, a LAN, a memory stick, a rented disk— and moves bytes.
- Layer 2 yields a useful analogy: what a wire does in the ISO/OSI model, namely decide who is adjacent to whom, is here decided by the possession of key material, so that the network is *dematerialised* and no service can occupy a privileged position in it.
- Layer 3 names a hub by a public key derived from a shared secret.
- At Layer 4, using a symmetric exchange protocol, replicas exchange messages about content-addressed blocks and converge by set union. In this model, data do not flow along a connection as they do in TCP or UDP, they accumulate as replicas merge.
- Layers 5 and 6 provide the Nearbytes APIs: the hubs, that is the append-only partially ordered logs —which will be taken as communication media in Section 3— and the replay mechanisms —which extract the current state from their history following an event-sourcing pattern.

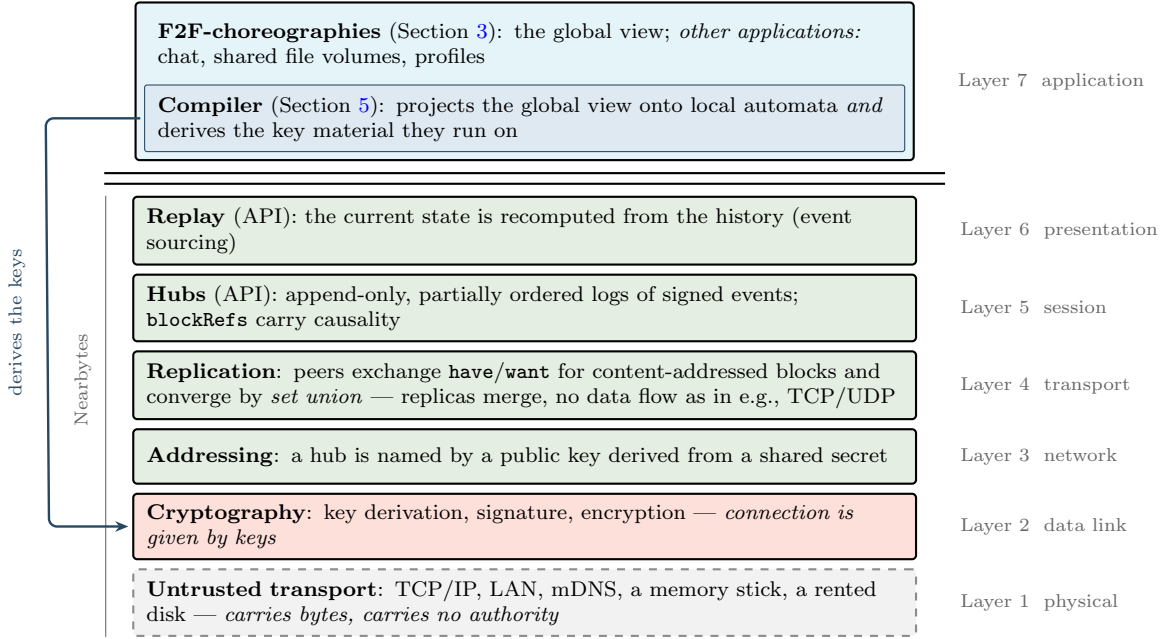


Figure 2: The Nearbytes stack, in the style of the ISO/OSI reference model, with the F2F-choreographies of this paper as an example of application level.

These six bottom-most layers constitute the platform of Nearbytes; Layer 7, separated by the double line from Nearbytes, is the application level. The F2F choreographic model of this paper sits at this level. The edge connecting our compiler with Layer 2 represents that we derive the cryptographic infrastructure needed by Nearbytes from F2F choreographies. This is why our formalisation in Sections 3 and 4 can abstract away from cryptography altogether.

2.2.3. The API of Nearbytes

Let us now survey the API of Nearbytes; the interested reader is referred to [17]. The API provides a data-centric, F2F infrastructure that permits local-first coordination [1] through reliable decentralised data storage mechanisms, which is also resilient to network partitioning. This API enables fully decentralised data sharing on top of which one can build application-level protocols featuring local-first coordination and confidential communication (such as file systems, chat, or profile and identity management). Conceptually, the framework can be thought of as a decentralised, shared *state* ensuring identity certification, authorisation, authentication, and other security properties by cryptographic means.

Data content is addressed by hash, as usual in friend-to-friend systems, with the addition of the mechanisms for addressing and updating the state sketched in Section 2.2.1. A read-write log, called *hub* in Nearbytes terminology, can be addressed in the “lake” of encrypted blocks through *deterministic key-pair generation* which allows users to generate a cryptographic infrastructure once a user-chosen secret is established. This mechanism lets a group of agents independently generate a public-private key-pair and a symmetric key out of a shared secret. The generated public key is used to address the hub, the corresponding private key to sign messages belonging to the hub, and the symmetric key to ensure confidentiality.

Messages have an optional clear-text part used to indicate *block references* to hashes of other messages or data blocks, giving rise to a partial order. This is a distinguished feature of Nearbytes as it allows to locally reconstruct hubs exploiting an event-sourcing pattern mechanism.

As said, the API of Nearbytes is designed to support application level protocols using local-first friend-to-friend coordination featuring confidentiality. For instance, a group of friends might want to share a read-write file system on top of an opaque peer-to-peer storage network. In order to do so, they need to share a secret; since they are friends the secret can be established on one of their social media groups.

```

1  export interface SignedEvent {
2      readonly envelope                : EventEnvelope;
3      readonly signature                : Signature;
4  }

5  export interface EventEnvelope {
6      readonly publicKey                : string;
7      readonly blockRefs                : readonly Hash[];
8      readonly ciphertext                : EncryptedData;
9  }

10 interface EventLogApi {
11     storeEvent(publicKey, signedEvent) : EventHash;
12     listEvents(publicKey)               : EventHash[];
13     retrieveEvent(publicKey, eventHash) : SignedEvent;
14 }
...

```

Figure 3: Nearbytes Log API v1 (persistence contract)

The secret can then be used to write commands such as “create file”, “delete file”, “create directory”, “rename directory”, “write block”, over the log, using the last seen block as a block reference. As typical in local-first coordination, consistency can break; hence when inconsistent events occur the group can decide how to handle divergence. For instance, the group can either have a programmed policy to reconcile the forks at the application level (as e.g., done in git) or adopt known methods for tie-breaking (as done e.g. in [2] assuming Lamport’s timestamps, or trusting timestamps to a certain degree, or else by employing a hierarchy of authority among agents, etc.).

In the stack of Nearbytes (cf. Section 2.2.2), hub repositories are materialised by the `nearbytes-log` package, which implements the normative Log API, a snippet of which is given in Fig. 3. A `SignedEvent` is a pair consisting of an event `envelope` and a `signature` (lines 1-4 in Fig. 3); the former bears the public key to verify authenticity, a cleartext list of references to other blocks, and a ciphertext, carrying application-level payload (lines 5-9). A reference inside an event can refer to either an existing event or a content block by means of a hash digest, establishing a partial order of causal dependencies.² On top of such a causal log, complex application-level protocols are possible (and a number are implemented); we describe the API on lines 10-15 of Fig. 3 and illustrate its usage to create F2F applications through a simple, yet realistic, scenario considering a hub for collaborative work where a group of users edit a file, say `foo.txt`.

Firstly, the agents agree on a secret used by Nearbytes to generate the symmetric and public-private keys. Adopting the notation of Section 2.2.1, the generated keys are a symmetric key `K` (for ensuring confidentiality) and a public-private key-pair, say `Kedit+` and `Kedit-`; the former key is used to address the hub while the latter key is used to ensure authentication and authenticity. The first action in our scenario is the initialisation of `foo.txt`; this can be done through two signed events which we now create and store in the hub. The payload of the first event is

```

payload1 = nb_block_1, nb_block_2, event1           where
nb_block_1 = {create_file(foo.txt)}K
nb_block_2 = {create_file}Kedit+
event1 = {[], hash(nb_block_1), hash(nb_block_2)}Kedit-

```

Notice that `create_file` and `contents` (used below) are not functions to be invoked, rather they are application-level “commands”, namely plain strings that the application interprets as expected. The sec-

²Nearbytes also offers a block storage API, to store (encrypted) raw payloads for data contents, which we omit for simplicity. For the sake of this paper, it suffices to know that it’s not mandatory to store payloads in events.

248 and block carries the *type* of the message, that is an observable abstraction of its content; here we take
 the command deprived of its arguments, so that the type of `create_file(foo.txt)` is `create_file` and,
 250 likewise, the type of `contents(foo.txt, A)` below is `contents`. Note that `event1` has an empty list of
 block references, since it is the first event of the hub. The invocation to the `storeEvent` operation from the
 252 `EventLogAPI` returns a hash digest, say H_1 , univocally identifying `event1` in the hub. A second payload,
 initialising the contents of `foo.txt` to A , is then stored; its event refers to H_1 :

```

    payload2 = nb_block_3, nb_block_4, event2           where
    nb_block_3 = {contents(foo.txt, A)}K
    254 nb_block_4 = {contents}Kedit+
    event2 = {[ $H_1$ ], hash(nb_block_3), hash(nb_block_4)}Kedit-
  
```

Assume that H_2 is the hash digest of `event2`. Later, a third payload updating `foo.txt` with contents B
 256 can be added to the log; it is obtained from the one above by replacing A with B in `nb_block_3` and $[H_1]$
 with $[H_2]$ in the block references list of the event.

258 Noteworthy, the execution above is fully *local*! The hub created is initially accessible only on the node
 where the steps above have been executed. The other users can access the hub at a later stage and reconstruct
 260 its state by replaying the log of events and applying changes to their own local replica of the hub. As said,
 conflicts are possible; for instance another user may issue an event carrying a different update of `foo.txt`
 262 than B . This is an application-level concern that can be solved in many ways, and is orthogonal to the
 event-sourcing pattern used to store commands, which is, instead, conflict-free by the cryptographic nature
 264 of the hashing and encryption primitives.

3. Friend-to-Friend Choreographies

266 We build a model to specify the global view of a choreographic abstraction tailored to local-first coordi-
 nation in friend-to-friend contexts. To this end we introduce the following syntax.

268 **Definition 7.** A friend-to-friend choreography (*F2F-choreography*, for short) is a term derived from the
 following grammar:

$$F ::= A \rightsquigarrow B : \mathbf{t} \mid F; F' \mid F \mid F' \mid F + F'$$

270 where B is a finite (possibly empty) set of buddies, namely agent identifiers, that A intends to interact with.

Essentially, the syntax of F2F-choreographies is obtained by replacing the interaction of g-choreographies
 272 with one allowing a finite set of receivers instead of a single receiver.³ As for g-choreographies, we abstract
 from the information exchanged in the interaction, which is simply represented by a message type $\mathbf{t}$. In the
 274 following, we may write $A \rightsquigarrow B : \mathbf{t}$ and $A \rightsquigarrow \mathbf{t}$ when B is the singleton $\{B\}$ and the empty set, respectively.

Besides the one-to-many interaction allowing interactions among several agents, another key feature of
 276 F2F-choreographies is that they enable agents to observe interactions they are not involved with.⁴ This is
 the main difference between F2F-choreographies and g-choreographies. In fact, the use of local shared logs
 278 combined with our semantics of the sequential composition establishes implicit inter-dependencies among
 agents of an F2F-choreography which classical choreographic models lack.

280 The semantics of F2F-choreographies can be given in terms of partial orders on the events they represent,
 like done for the semantics of g-choreographies. A key design choice is to identify hubs with the pomsets
 282 capturing the behaviour of F2F-choreographies. Essentially, we label events with interactions form $A!B(\mathbf{t})$
 (generalising the output label of g-choreographies) to represent the fact that A intends to share a message
 284 of type $\mathbf{t}$ with the buddies in B . This implies that hubs are *monotonically increasing* structures: agents can
 verify if some event occurred, but they cannot remove the event from the hub.

³As for g-choreographies, we consider only finitary terms for simplicity.

⁴As we will see, only the meta-information of the event is accessible to agents other than A or those in B .

Similarly to the semantics of g-choreographies, the semantics of an F2F-choreography F is a set of pomsets $\llbracket F \rrbracket$, one for each branch of the choices in F . The following auxiliary operator which intuitively makes some events of a pomset become immediate causes of events in a subsequent pomset will be useful for the next definition: given two pomsets r and r' with disjoint sets of events (namely, $\mathcal{E}_r \cap \mathcal{E}_{r'} = \emptyset$) and a set of events $E \subseteq \mathcal{E}_r$, let

$$r;_E r' = [\mathcal{E}_r \cup \mathcal{E}_{r'}, \hookrightarrow_r \cup \hookrightarrow_{r'} \cup E \times \mathcal{E}_{r'}, \lambda']$$

where $\lambda' = \lambda_r[e' \mapsto \lambda_{r'}(e')] \mid e' \in r'$

Definition 8. The semantics of an F2F-choreography F is defined by induction on the syntactic structure of F as follows:

$$\begin{aligned} \llbracket A \rightsquigarrow B : t \rrbracket &= \left\{ \llbracket A!B\langle t \rangle \rrbracket \right\} \\ \llbracket F + F' \rrbracket &= \llbracket F \rrbracket \cup \llbracket F' \rrbracket \\ \llbracket F \mid F' \rrbracket &= \{ r;_{\emptyset} r' \mid r \in \llbracket F \rrbracket \text{ and } r' \in \llbracket F' \rrbracket \text{ and } \mathcal{E}_r \cap \mathcal{E}_{r'} = \emptyset \} \\ \llbracket F; F' \rrbracket &= \{ r;_{\mathcal{E}_r} r' \mid r \in \llbracket F \rrbracket \text{ and } r' \in \llbracket F' \rrbracket \text{ and } \mathcal{E}_r \cap \mathcal{E}_{r'} = \emptyset \} \end{aligned}$$

We abbreviate $A!\emptyset\langle t \rangle$ and $A!\{B\}\langle t \rangle$ respectively with $A!\langle t \rangle$ and $A!B\langle t \rangle$.

Notice that the use of the $_{;\emptyset}$ operator in the semantics of the parallel composition $F \mid F'$ introduces no new dependencies among the events of F and F' , so it is consistent with the intuition that the two F2F-choreographies proceed independently of each other. Also, in case of sequential composition $F; F'$ in Definition 8, all events of the first choreography F are ordered before events of F' independently of the agents involved in F and F' . This follows from a deliberate modelling assumption according to our definition of payload (cf. Section 2.2.1): messages' content is confidential, but agents can coordinate by observing public meta-data of interactions, such as their occurrence, public type, and causality in a hub, even when they are not explicit recipients. This captures a coordination setting where progress may depend on awareness that some event has been published, while the confidential content of that event remains restricted to the intended recipients. An alternative semantics would order events across $F; F'$ only when there is an explicit agent overlap (i.e., events have the same sender, or the receivers of an event in F include the sender of an event in F'). We do not adopt such a stricter causal interpretation here, since our goal is to model protocols where later actions may be enabled by the publicly observable earlier event types rather than by accessing their content. The next example shows this key feature of F2F-choreographies: agents observe interactions they are not directly involved with.

Example 9. The F2F-choreography $A \rightsquigarrow C : t_1; B \rightsquigarrow t_2$ is meaningful (i.e., it can be implemented distributively) since agent B can observe the interaction between A and C despite the output event corresponding to the interaction with message type t_1 targets C only.

This implies that the semantics of $F = (A \rightsquigarrow C : t_1; B \rightsquigarrow t_2) + (A \rightsquigarrow t_3; C \rightsquigarrow t_2 \mid D \rightsquigarrow t_2)$ is

$$\llbracket F \rrbracket = \left\{ \left[\begin{array}{c} A!C\langle t_1 \rangle \\ \downarrow \\ B!\langle t_2 \rangle \end{array} \right] , \quad \left[\begin{array}{c} A!\langle t_3 \rangle \\ \swarrow \quad \searrow \\ C!\langle t_2 \rangle \quad D!\langle t_2 \rangle \end{array} \right] \right\}$$

whose two pomsets correspond to the two branches of the F2F-choreography F . ◇

Definition 10. The language of an F2F-choreography F is the set of linearisations of the pomsets in $\llbracket F \rrbracket$, namely the permutations of all the events' labels that preserve the order relations imposed by the pomsets.

Example 11. The language of the F2F-choreography F in Example 9 consists of three words: word $A!C\langle t_1 \rangle \cdot B!\langle t_2 \rangle$ for the first pomset and the two $A!\langle t_3 \rangle \cdot C!\langle t_2 \rangle \cdot D!\langle t_2 \rangle$ and $A!\langle t_3 \rangle \cdot D!\langle t_2 \rangle \cdot C!\langle t_2 \rangle$ for the second one, since events from C and D are not ordered. ◇

4. A Nearbytes Inspired Local Model

We now develop a formal framework where *agents* can confidentially exchange information in a *fully decentralised* way, in other words, we define the local views of our choreographic model. Interestingly, we follow the pattern used in g-choreographies by replacing communicating finite-state machines with a new class of automata whose transitions carry enough information to realise such automata on top of Nearbytes. In the following we assume that, together with their content, messages carry some meta-information that we will spell out as we go along.⁵ In particular, we keep the type $\mathbf{t}$ in a block of its own; this is what allows an agent to decide *whether* an event concerns it, that is, whether the event triggers a local state transition, by retrieving and decrypting a small, constant-size block, without ever fetching the content, which may be arbitrarily large. Hereafter, we assume that messages are typed and uniquely identifiable and that they specify a (possibly empty) set of (identifiers of) predecessors⁶. Actually, messages are bijectively associated with communication events which are formalised as *fresh* names that will be used to build (the partial orders forming) hubs; therefore hereafter we use ‘message’ and ‘event’ interchangeably.

Summing up: in our model hubs are repositories of messages uniquely identified by secrets (cf. Section 2.2.1); intuitively, a secret $\mathbf{h}$ is the address of a repository known only to agents that can share information and coordinate via the hub (identified by) $\mathbf{h}$ as well as other hubs if necessary. As said, each message carries the information about its predecessors; this implies that hubs form pomsets. For simplicity, we assume that hubs cannot be dynamically created and are statically known to agents.

Agents’ behaviour is modelled by a symbolic automaton⁷ (S, s_0, δ) where, as usual, S is a finite set of states with s_0 being the initial state, and δ is a transition relation; we write $\delta : s \xrightarrow{\ell} t$ when there is a transition from s to t with label ℓ in δ and simply write $s \xrightarrow{\ell} t$ when δ is irrelevant or clear from the context.

Definition 12. A label ℓ represents an operation on a hub and has the following shape:

- an output action $\mathbf{h}!\mathcal{B} \cdot \underline{b} \hookrightarrow (a : \mathbf{t})$ stores a message of type $\mathbf{t}$ meant for the agents in $\mathcal{B}$ on hub $\mathbf{h}$; the execution of the output action implicitly assigns a with the unique identity of the message; finally, the event will immediately follow the messages identified by the identifiers in $\underline{b}$;⁸
- an input action $\mathbf{h}?(a : \mathbf{t})@\mathbf{A}$ reads a message of type $\mathbf{t}$ that $\mathbf{A}$ sent on hub $\mathbf{h}$ and assigns it to the identifier a ;
- a query action *inspects a condition on the hub*: $\mathbf{h} \odot a \hookrightarrow b$ checks that a is a cause of b , $\mathbf{h} \odot a \nrightarrow b$ checks that b is not dependent on a , and $\mathbf{h} \odot \hat{a} \subseteq \underline{b}$ checks that $\underline{b}$ includes the immediate causes of a .

Names of actions are free if they are not surrounded by round brackets (such as the names in $\underline{b}$ in the label $\mathbf{h}!\mathcal{B} \cdot \underline{b} \hookrightarrow (a : \mathbf{t})$) and bound otherwise (such as a in $\mathbf{h}?(a : \mathbf{t})@\mathbf{A}$).

The firing of an output transition adds a new event corresponding to the label of the transition in the hub; contextually, the event’s immediate causes are recorded as specified by the output action. The actual behaviour of agents is determined by a renaming map that assigns local variable for the placeholders; for instance, the firing of a transition labelled by an output action $\mathbf{h}!\mathcal{B} \cdot \underline{b} \hookrightarrow (a : \mathbf{t})$ maps the placeholder a to the freshly generated event corresponding to the action (i.e., the event added to the hub with the sent message).

Definition 13. A configuration is a tuple $\sigma \cdot \rho \cdot \nu \cdot \mathbf{H}$ where

⁵At the implementation level, payloads can be any (serialised) data structures; our model abstracts away payloads since they are inconsequential for our purposes.

⁶In Section 2.2.1 this meta-information is carried by the blocks **event**. Also, our paper is oblivious of the actual cryptographic algorithms; as usually done in settings like ours, we work under the perfect encryption hypothesis.

⁷We admit automata with cycles; however, given our simplifying assumption that F2F-choreographies are finite, in the paper we use only acyclic automata.

⁸When $\mathcal{B}$ or $\underline{b}$ are the empty set we simply omit them and write e.g., $\mathbf{h}! \cdot \underline{b} \hookrightarrow (a : \mathbf{t})$, or $\mathbf{h}!\mathcal{B}(a : \mathbf{t})$, or $\mathbf{h}!(a : \mathbf{t})$; also, we may omit curly brackets when such sets are singletons.

$$\begin{array}{c}
\frac{\sigma(\mathbf{A}) \xrightarrow{\mathbf{h}!\mathcal{B} \cdot \underline{b} \hookrightarrow (a:\mathbf{t})} s' \quad e \notin r = \mathbf{H}(\mathbf{h}) \quad \mathbf{v} = \nu(\mathbf{A})[a \mapsto e] \quad E = \nu(\mathbf{A})(\underline{b})}{\sigma \cdot \rho \cdot \nu \cdot \mathbf{H} \xrightarrow{\mathbf{A}!\mathcal{B}(\mathbf{t})} \sigma[\mathbf{A} \mapsto s'] \cdot \rho \cdot \nu[\mathbf{A} \mapsto \mathbf{v}] \cdot \mathbf{H}[\mathbf{h} \mapsto r;_E [\{e\}, \emptyset, e \mapsto \mathbf{A}!\mathcal{B}(\mathbf{t})]]} \\
\\
\frac{\sigma(\mathbf{A}) \xrightarrow{\mathbf{h}?(a:\mathbf{t})@\mathbf{B}} s' \quad r = \mathbf{H}(\mathbf{h}) \quad e \in \mathcal{E}_r \setminus \rho(\mathbf{A}) \quad \lambda_r(e) = \mathbf{B}!\mathcal{B}(\mathbf{t}) \quad \mathbf{v} = \nu(\mathbf{A})[a \mapsto e]}{\sigma \cdot \rho \cdot \nu \cdot \mathbf{H} \xrightarrow{\tau} \sigma[\mathbf{A} \mapsto s'] \cdot \rho[\mathbf{A} \mapsto \rho(\mathbf{A}) \cup \{e\}] \cdot \nu[\mathbf{A} \mapsto \mathbf{v}] \cdot \mathbf{H}} \\
\\
\frac{\sigma(\mathbf{A}) \xrightarrow{\ell} s' \quad \mathbf{H}, \mathbf{A}, \nu \models \ell}{\sigma \cdot \rho \cdot \nu \cdot \mathbf{H} \xrightarrow{\tau} \sigma[\mathbf{A} \mapsto s'] \cdot \rho \cdot \nu \cdot \mathbf{H}}
\end{array}$$

Figure 4: Semantics of the local view

- σ maps each agent to the current state in its automaton;
- ρ maps agents to the set of messages that they have already read;
- ν is an environment assigning to each agent a function mapping a name identifier in the automaton to an event in the hub;
- $\mathbf{H}$ maps hubs to pomsets whose labelling function λ labels events with terms of the form $\mathbf{A}!\mathcal{B}(\mathbf{t})$.

A configuration $\sigma \cdot \rho \cdot \nu \cdot \mathbf{H}$ is initial if for all agents $\mathbf{A}$, $\sigma(\mathbf{A})$ is the initial state of the automaton of $\mathbf{A}$, $\rho(\mathbf{A}) = \emptyset$, and $\nu(\mathbf{A})$ is the empty map.

Since we deal with symbolic automata, we have to set some well-formedness conditions to avoid meaningless computations. For instance, if $\underline{b}$ is a non-empty set of names, the automaton

$$\rightarrow s \xrightarrow{\mathbf{h}!\mathcal{B} \cdot \underline{b} \hookrightarrow (a:\mathbf{t})} s' \xrightarrow{\mathbf{h}!\mathcal{B}(a:\mathbf{t})} s''$$

is not valid since in the initial configuration the environment will be undefined on the names in $\underline{b}$. We therefore constrain our automata by requiring that for each transition $s \xrightarrow{\ell} s'$ the free names in ℓ occur bound with the same message type in all the paths reaching s .

Definition 14. The semantics of our symbolic automata is the smallest relation on configurations induced by the rules in Fig. 4 (the relation $\mathbf{H}, \mathbf{A}, \nu \models \ell$ used in the last rule is defined in Definition 15).

Let us spell the semantic rules in Fig. 4 which for convenience are repeated as we comment them.

An agent firing an output transition introduces in the configuration a fresh output event according to the first rule in Fig. 4:

$$\frac{\sigma(\mathbf{A}) \xrightarrow{\mathbf{h}!\mathcal{B} \cdot \underline{b} \hookrightarrow (a:\mathbf{t})} s' \quad e \notin r = \mathbf{H}(\mathbf{h}) \quad \mathbf{v} = \nu(\mathbf{A})[a \mapsto e] \quad E = \nu(\mathbf{A})(\underline{b})}{\sigma \cdot \rho \cdot \nu \cdot \mathbf{H} \xrightarrow{\mathbf{A}!\mathcal{B}(\mathbf{t})} \sigma[\mathbf{A} \mapsto s'] \cdot \rho \cdot \nu[\mathbf{A} \mapsto \mathbf{v}] \cdot \mathbf{H}[\mathbf{h} \mapsto r;_E [\{e\}, \emptyset, e \mapsto \mathbf{A}!\mathcal{B}(\mathbf{t})]]}$$

specifying the firing of an output transition $\sigma(\mathbf{A}) \xrightarrow{\mathbf{h}!\mathcal{B} \cdot \underline{b} \hookrightarrow (a:\mathbf{t})} s'$ from the current state $\sigma(\mathbf{A})$ of an agent $\mathbf{A}$; this extends the hub with a new immediate consequence $e \notin r = \mathbf{H}(\mathbf{h})$ of the events specified in the output action (that is the events assigned to the names in $\underline{b}$ by $\nu(\mathbf{A})$). Note that the new output is uniquely identified by a fresh event e assigned to a .

Message reception is non-deterministic and, according to the rule below, receivers can read consecutive messages in the opposite order of arrival:

$$\frac{\sigma(\mathbf{A}) \xrightarrow{\mathfrak{h}?(a : \mathbf{t})@B} s' \quad r = H(\mathfrak{h}) \quad e \in \mathcal{E}_r \setminus \rho(\mathbf{A}) \quad \lambda_r(e) = B!\mathcal{B}\langle \mathbf{t} \rangle \quad \mathbf{v} = \nu(\mathbf{A})[a \mapsto e]}{\sigma \cdot \rho \cdot \nu \cdot H \xrightarrow{\tau} \sigma[\mathbf{A} \mapsto s'] \cdot \rho[\mathbf{A} \mapsto \rho(\mathbf{A}) \cup \{e\}] \cdot \nu[\mathbf{A} \mapsto \mathbf{v}] \cdot H}$$

Intuitively, the semantics selects an event e from the pomset r (e.g., $e \in \mathcal{E}_r$) whose label $\lambda_r(e) = B!\mathcal{B}\langle \mathbf{t} \rangle$ matches the input action, namely an event e generated by an output from B of a message of type $\mathbf{t}$. Notice that the condition $e \notin \rho(\mathbf{A})$ in the premise of the rule prevents e from being received twice; intuitively, known events are “no longer interesting” and this allows receivers to distinguish different occurrences of the same type (for instance, when agents have to react after the second occurrence of an event of some type). Firing the input transition updates the map of the receiver to $\mathbf{v} = \nu(\mathbf{A})[a \mapsto e]$, namely the placeholder a gets assigned with the read event e ; likewise, we have that the set of the messages received by $\mathbf{A}$ becomes $\rho(\mathbf{A}) \cup \{e\}$.

Finally, the third rule in Fig. 4

$$\frac{\sigma(\mathbf{A}) \xrightarrow{\ell} s' \quad H, \mathbf{A}, \nu \models \ell}{\sigma \cdot \rho \cdot \nu \cdot H \xrightarrow{\tau} \sigma[\mathbf{A} \mapsto s'] \cdot \rho \cdot \nu \cdot H}$$

handles query actions by simply updating the current state of $\mathbf{A}$ when the configuration satisfies the query action according to the relation given in the next definition.

Definition 15. An assignment H , an agent $\mathbf{A}$, and an environment ν satisfy a label ℓ , written $H, \mathbf{A}, \nu \models \ell$, when

$$\begin{aligned} \ell = \mathfrak{h} @ a \hookrightarrow b &\implies \nu(\mathbf{A})(a) \hookrightarrow_{H(\mathfrak{h})} \nu(\mathbf{A})(b) \\ \ell = \mathfrak{h} @ a \not\hookrightarrow b &\implies \nu(\mathbf{A})(a) \not\hookrightarrow_{H(\mathfrak{h})} \nu(\mathbf{A})(b) \\ \ell = \mathfrak{h} @ \widehat{a} \subseteq \widehat{b} &\implies \widehat{\nu(\mathbf{A})(a)} \subseteq \{ \nu(\mathbf{A})(b) \mid b \in \widehat{b} \} \end{aligned}$$

(where $\hookrightarrow_{_}$ and $\widehat{_}$ are defined in Definition 3).

The next examples illustrate our semantics using some simple cases.

Example 16. Let agents $\mathbf{A}$ and $\mathbf{B}$ respectively behave according to the following automata

$$s_{\mathbf{A}}^1 \xrightarrow{\mathfrak{h}!\mathcal{B}(a : \mathbf{t}_1)} s_{\mathbf{A}}^2 \xrightarrow{\mathfrak{h}!\mathcal{B} \cdot \underline{a} \hookrightarrow (b : \mathbf{t}_2)} s_{\mathbf{A}}^3 \quad \text{and} \quad s_{\mathbf{B}}^1 \xrightarrow{\mathbf{A}?(a : \mathbf{t}_2)@ \mathfrak{h}} s_{\mathbf{B}}^2 \xrightarrow{\mathbf{A}?(b : \mathbf{t}_1)@ \mathfrak{h}} s_{\mathbf{B}}^3$$

In an initially empty hub $H(\mathfrak{h})$ agent $\mathbf{A}$ produces two events, the first of type $\mathbf{t}_1$ and the second of type $\mathbf{t}_2$. When the first transition fires, a new event (say e) is generated. This makes the pomset of $H(\mathfrak{h})$ contain the singleton event e labelled $\mathbf{A}!\mathcal{B}\langle \mathbf{t}_1 \rangle$ and $\nu(\mathbf{A}) : a \mapsto e$. In the resulting configuration, $\mathbf{B}$ cannot therefore read an event of type $\mathbf{t}_2$ since none has been produced yet. However, (the automaton of) $\mathbf{A}$ moves to state $s_{\mathbf{A}}^2$ whose output transition generates a fresh event $e' \neq e$ of type $\mathbf{t}_2$; accordingly pomset $H(\mathfrak{h})$ is extended with e' , labelled by $\mathbf{A}!\mathcal{B}\langle \mathbf{t}_2 \rangle$. Notice that the event in the pomset relation resulting after the execution of the second output records that e is an immediate cause of e' .

Now $\mathbf{B}$ can fire its first transition; indeed, there is an event e' with label $\mathbf{A}!\mathcal{B}\langle \mathbf{t}_2 \rangle$ not yet read by $\mathbf{B}$ ($e' \notin \rho(\mathbf{B})$). Its mapping $\nu(\mathbf{B})$ is updated correspondingly and $\rho(\mathbf{B})$ is extended with e' . Finally, $\mathbf{B}$ can fire its second transition, extending the mapping $\nu(\mathbf{B})$ to map b to e . Notice that the two agents have different (and opposite) mappings for a and b . $\diamond$

Let us continue with an example showing how query actions fire.

Example 17. Suppose that s_B^3 in Example 16 has a transition labelled by a query action ℓ , then

- $\ell = \mathbf{h} \odot a \hookrightarrow b$ cannot fire since $\nu(\mathbf{B})(a) = e'$ and, in $H(\mathbf{h})$, event e' does not precede event $e = \nu(\mathbf{B})(b)$;
- $\ell = \mathbf{h} \odot b \hookrightarrow a$ can fire, since there the event $\nu(\mathbf{B})(a)$ is a consequence of the event $\nu(\mathbf{B})(b)$;
- $\ell = \mathbf{h} \odot a \not\hookrightarrow b$ can fire, since there the event $\nu(\mathbf{B})(a)$ does not precede the event $\nu(\mathbf{B})(b)$;
- $\ell = \mathbf{h} \odot \hat{a} \subseteq b$ can fire, since the predecessors of $\nu(\mathbf{B})(a)$ are within $\{\nu(\mathbf{B})(b)\}$;
- $\ell = \mathbf{h} \odot \hat{a} \subseteq \emptyset$ cannot fire, since the set of predecessors of $\nu(\mathbf{B})(a)$ is not empty;
- $\ell = \mathbf{h} \odot \hat{b} \subseteq \emptyset$ can fire, since the set of predecessors of $\nu(\mathbf{B})(b)$ is empty. $\diamond$

5. Projecting Friend-to-Friend Choreographies

As is commonplace in choreographic modelling, we define a *projection* operation that, given an F2F-choreography F and an agent $\mathbf{A}$, “compiles” F in an automaton specifying the expected behaviour of $\mathbf{A}$ in F . A distinguishing aspect with respect to usual projection operations, is that our projection aims to identify the minimal set of agents that should be aware of an interaction. For this reason, we characterise a hub as the minimal set of agents that can perform actions on the hub (and, therefore, that should know the secret to access the hub). If fewer secrets are desirable, one can simply take the union of sets of agents. The motivation behind this design choice is to avoid introducing unnecessary dependencies among agents.

We now define the projection operation. To avoid technical intricacies, we assume that each message type can appear only on nodes of the same branch of the parse tree of the choreography. (We will discuss this further in Section 9.) Our definition is parametric on the hub-characterising set, namely besides the choreography F and the agent $\mathbf{A}$, the projection operation $F|_{\mathbf{A}}^{\mathcal{B}}$ takes a set $\mathcal{B}$ of agents enabled to operate on the hub.

Definition 18. The projection of an F2F-choreography F with respect to an agent $\mathbf{A}$ is the automaton $F|_{\mathbf{A}}^{\emptyset}$ where the projection operation $F|_{\mathbf{A}}^{\mathcal{B}}$ is inductively defined in the rest of the section.

The projection of an interaction produces an automaton with a single output transition of the sender:

$$\mathbf{A} \rightsquigarrow \mathcal{B} : \mathbf{t}|_{\mathbf{A}}^{\mathcal{B}'} = (\{s_0, s\}, s_0, \{s_0 \xrightarrow{\{\mathbf{A}\} \cup \mathcal{B} \cup \mathcal{B}'! \mathcal{B}(a:\mathbf{t})} s}\})$$

For any other agent $\mathbf{B} \neq \mathbf{A}$, the projection is an automaton with a receiving transition if $\mathbf{B}$ is interested in the event (i.e., $\mathbf{B} \in \mathcal{B} \cup \mathcal{B}'$),

$$\mathbf{A} \rightsquigarrow \mathcal{B} : \mathbf{t}|_{\mathbf{B}}^{\mathcal{B}'} = \begin{cases} (\{s_0, s\}, s_0, \{s_0 \xrightarrow{\{\mathbf{A}\} \cup \mathcal{B} \cup \mathcal{B}'?(a:\mathbf{t})@ \mathbf{A}} s}\}) & \text{if } \mathbf{B} \in \mathcal{B} \cup \mathcal{B}' \\ (\{s_0\}, s_0, \emptyset) & \text{otherwise} \end{cases}$$

otherwise, the projection is single state automaton with no transition:

We now define the projection on composite choreographies starting from sequential composition for which we use the auxiliary notion of *ready set* of a choreography that is the set of agents that can start an interaction:

Definition 19. The ready set of an F2F-choreography F is the set $\text{RS}(F)$ defined by

- $\text{RS}(\mathbf{A} \rightsquigarrow \mathcal{B} : \mathbf{t}) = \{\mathbf{A}\}$
- $\text{RS}(F \mid F') = \text{RS}(F) \cup \text{RS}(F')$
- $\text{RS}(F + F') = \text{RS}(F) \cup \text{RS}(F')$
- $\text{RS}(F; F') = \text{RS}(F)$

Let $(\hat{S}, \hat{s}, \hat{\delta}) = F|_{\mathbf{A}}^{\text{RS}(F')}$ and $(S', i', \delta') = F'|_{\mathbf{A}}^{\mathcal{B}}$ with $\hat{S} \cap S' = \emptyset$. We set

$$F; F'|_{\mathbf{A}}^{\mathcal{B}} = (\hat{S} \cup S', s_0, \{s \xrightarrow{\ell} s' \mid s \text{ has no transition in } \delta \text{ and } \delta' : s'_0 \xrightarrow{\ell} s'\} \cup \delta \cup \delta')$$

Before giving the definition in the other cases, let us give an example of faithful projection.

Example 20. Consider the choreography $A \rightsquigarrow C : t_1; B \rightsquigarrow t_2$. The projection $(B \rightsquigarrow t_2)|_B^\emptyset$ yields the two-states automaton $q_0^B \xrightarrow{\{B\}!(a:t_2)@B} q_1^B$ (note that there are no receivers or immediate successors). For every other agent X , $(B \rightsquigarrow t_2)|_X^\emptyset$ is an automaton with a single state s_0^X and no transitions.

The projection of the sequential composition of $A \rightsquigarrow C : t_1$ with $B \rightsquigarrow t_2$ uses $RS(B \rightsquigarrow t_2) = \{B\}$, hence we have that $(A \rightsquigarrow C : t_1)|_A^{\{B\}}$ is the two-states automaton $s_0^A \xrightarrow{\{A,B,C\}!C(a:t_1)} s_1^A$. Notice that in this case we use a hub that is accessible by A , B , and C since B must be aware of the termination of this choreography and C is the explicit recipient of the message. Moreover, $(A \rightsquigarrow C : t_1)|_X^{\{B\}}$ is the two-states automaton $s_0^X \xrightarrow{\{A,B,C\}?(a:t_1)@A} s_1^X$ for $X \in \{B, C\}$ and a single-state automaton with no transitions for any agent X other than A , B , or C .

The sequential composition connects these automata possibly as follows:

- for A we have the automaton $s_0^A \xrightarrow{\{A,B,C\}!C(a:t_1)} s_1^A$;
- for B , after q_0^B replacing with s_1^B in $(B \rightsquigarrow t_2)|_B^\emptyset$, we have $s_0^B \xrightarrow{\{A,B,C\}?(a:t_1)@A} s_1^B \xrightarrow{\{B\}!(a:t_2)@B} q_1^B$;
- for C we get $s_0^C \xrightarrow{\{A,B,C\}?(a:t_1)@A} s_1^C$;
- and for any other X it simply returns an automaton with a single state s_0^X and no transition.

◇

The projection of parallel composition simply produces an automaton where the transitions are interleaved; let $(S, s_0, \delta) = F|_A^B$ and $(S', s'_0, \delta') = F'|_A^B$, then

$$(F \mid F')|_A^B = (S \times S', (s_0, s'_0), \{(s, s') \xrightarrow{\ell} (t, t') \mid \delta : s \xrightarrow{\ell} t, s' \in S'\} \cup \{(s, s') \xrightarrow{\ell} (s, t') \mid \delta' : s' \xrightarrow{\ell} t', s \in S\})$$

Namely, the projection of a parallel composition is the product of the automata of the projections of the components.

For choice we add to the initial state of the first projection the initial transitions of the second one:

$$(F + F')|_A^B = (S \cup S' \setminus \{s'_0\}, s_0, \delta \cup \delta'')$$

where δ'' is obtained by replacing s'_0 with s_0 in all the transitions of δ' .

Note that the projection operation yields automata with a single initial state and a single state with no outgoing transitions.

The next example builds on Example 20 to show the other cases of the projection operation.

Example 21. Let us consider the projections of $F + F'$ where F is the F2F-choreography in Example 20 and $F' = A \rightsquigarrow t_3; C \rightsquigarrow t_2 \mid D \rightsquigarrow t_2$. We have that

- $F'|_A^\emptyset$ is the automaton $s_0^A \xrightarrow{\{A,C,D\}!C(a:t_3)} s_1^A$;
- $F'|_B^\emptyset$ is the automaton s_1^B ;
- $F'|_C^\emptyset$ is the automaton $s_0^C \xrightarrow{\{A,C,D\}?(a:t_3)@A} s_1^C \xrightarrow{\{C\}!(a:t_2)@C} s_3^C$;
- $F'|_D^\emptyset$ is the automaton $s_0^D \xrightarrow{\{A,C,D\}?(a:t_3)@A} s_1^D \xrightarrow{\{D\}!(a:t_2)@D} s_3^D$.

Finally, the choice operator combines these automata by replacing the initial states of the last three ones with the initial state of $F'|_A^\emptyset$.

◇

Notice that the projection attempts to minimise the number of agents that must be notified of a message, therefore it avoids sharing unnecessary information. If one wants to reduce the number of hubs, they can simply group hubs by making the union of the agents that should access the individual hubs. For instance, in Example 20 we could join the hubs $\{A, B, C\}$ and $\{A, C, D\}$ forming the hub $\{A, B, C, D\}$, sharing both messages t_1 and t_3 from A on the same hub.

Finally, we give an example of a choreography that cannot be faithfully implemented in asynchronous distributed settings.

Example 22. An F2F-choreography of the shape $A \rightsquigarrow B : t_1 + B \rightsquigarrow B' : t_2$ is not realisable when $A \neq B$ since A and B may not agree on who should send the message. $\diamond$

6. An Approach to Proving Projections' Correctness

We formalise soundness of projection in terms of *visible traces*, namely sequences of labels decorating the runs starting from an initial configuration with empty pomsets and no processed messages from where τ are eliminated (i.e., τ is treated as the neutral element of sequence concatenation).

Our proof assumes the *well-formedness* condition given in the next definition.

Definition 23 (Well-formedness). *An F2F-choreography F is well-formed whenever*

1. *in each choice $F_1 + F_2$ there is a unique agent with unguarded outputs in both branches (formally, the ready sets of F_1 and F_2 are the same singleton); and*
2. *for each sender, a message type cannot occur in two different branches of a choice, in two different components of a parallel composition, or in both components of a sequential composition.*

Definition 24 (Soundness of projection). *Let F be an F2F-choreography. Let $\mathbb{L}$ be the prefix closure of the language of $\llbracket F \rrbracket$. The projection of F is sound whenever the set of its visible traces are included in $\mathbb{L}$.*

Intuitively, our soundness condition amounts to say that observable behaviour of the projected system falls within the one specified in the F2F-choreography.

Claim 1 (Soundness of projection). *If F is well-formed, then the projection of F is sound.*

Proof sketch. By induction on the structure of the choreography. The first well-formedness condition in Definition 23 ensures that each choice is controlled by a unique agent while the second condition prevents an agent from confusing branches of a choice, inputs from different parallel threads, or interactions on different components of a sequential composition. We briefly discuss the main inductive cases below.

Parallel composition. For $F = F_1 \mid F_2$, the language of $\llbracket F \rrbracket$ consists of the shuffling of the languages of F_1 and F_2 . By the induction hypothesis, the visible traces of the two projected systems are respectively included in these languages.

We first build a transition system T whose states are the pairs of configurations $\chi = ((\sigma_1 \cdot \nu_1 \cdot \rho_1 \cdot H_1), (\sigma_2 \cdot \nu_2 \cdot \rho_2 \cdot H_2))$ where, for $i \in \{1, 2\}$, configuration $(\sigma_i \cdot \nu_i \cdot \rho_i \cdot H_i)$ is reachable from the projections of F_i and

the transitions from χ are all the ones done in the projections; more precisely $\chi \xrightarrow{\ell} \chi'$ if, and only if,

either $(\sigma_1 \cdot \nu_1 \cdot \rho_1 \cdot H_1) \xrightarrow{\ell} (\sigma'_1 \cdot \nu'_1 \cdot \rho'_1 \cdot H'_1)$ in the projection of F_1 and $\chi' = ((\sigma'_1 \cdot \nu'_1 \cdot \rho'_1 \cdot H'_1), (\sigma_2 \cdot \nu_2 \cdot \rho_2 \cdot H_2))$

or $(\sigma_2 \cdot \nu_2 \cdot \rho_2 \cdot H_2) \xrightarrow{\ell} (\sigma'_2 \cdot \nu'_2 \cdot \rho'_2 \cdot H'_2)$ in the projection of F_2 and $\chi' = ((\sigma_1 \cdot \nu_1 \cdot \rho_1 \cdot H_1), (\sigma'_2 \cdot \nu'_2 \cdot \rho'_2 \cdot H'_2))$.

The set of visible traces of T is equivalent to the shuffling of the visible traces of the two projected systems by construction, and therefore it is included in the language of $\llbracket F \rrbracket$. It remains to show that T has the same visible language of the projection of F . This follows by exhibiting a bisimulation between states of the interleaving transition system and states of the projection of F of the form

$$((\sigma_1 \cdot \nu_1 \cdot \rho_1 \cdot H_1), (\sigma_2 \cdot \nu_2 \cdot \rho_2 \cdot H_2)) \sim (\sigma \cdot \nu \cdot \rho_1 \cup \rho_2 \cdot H_1 \cup H_2)$$

where σ is the map $A \mapsto (\sigma_1(A), \sigma_2(A))$ that maps each agent A to a state in the product automaton. Here, unions of received events and hubs are understood point-wise. Notice that, since the automata generated by our projection do not use pomset queries, we do not need to relate ν explicitly to ν_1 and ν_2 .

The only non-immediate case of the bisimulation concerns input transitions. Suppose that the projection

of F performs $\sigma(A) \xrightarrow{h?(a : t)@B} s'$. Assume that this transition originates from the projection of F_1 ; the case for F_2 is symmetric. The input consumes an event e from $H_1 \cup H_2$, and, by definition of the projection,

$\sigma_1(A) \xrightarrow{h?(a : t)@B} s_1$ and $s' = (s'_1, \sigma_2(A))$. To simulate this transition in the explicitly interleaved system, we must show that e belongs to H_1 rather than H_2 . This follows from the assumption that a message type occurs in only one of the two parallel threads for a given sender. Hence F_2 cannot have produced a matching event, and the first projected system can perform the same input. The bisimulation is therefore preserved.

Choice composition. For $F = F_1 + F_2$, the language of $\llbracket F \rrbracket$ is the union of the languages of F_1 and F_2 . We first build a transition system T obtained by adding the transitions of the initial configuration of the transition system of the projection of F_2 to the initial configuration of the transition system of the projection of F_1 . By construction, the visible language of T is the union of the visible languages of the two projected systems, which, by the induction hypothesis, is included in the union of the languages of the semantics of F_1 and F_2 .

We then relate T to the transition system obtained from the projection of F . Here we use the identity as bisimulation. The main point is to show that all agents consistently follow the same branch. More precisely,

suppose that A performs a transition $\sigma \cdot \nu \cdot \rho \cdot H \xrightarrow{\ell} \sigma' \cdot \nu' \cdot \rho' \cdot H'$ from a configuration in which $\sigma(A)$ is still an initial state, while there exists an agent B such that $\sigma(B)$ is non-initial and belongs to the projection of F_i . We must show that $\sigma'(A)$ also belongs to the projection of F_i .

This property follows from the well-formedness condition for choice. We require that the two branches do not share message types and that there is a unique agent C with unguarded outputs in both F_1 and F_2 . The argument proceeds by induction on the number of agents that have left their initial states.

In the base case, the first agent performing an output must be C , and its transition determines the selected branch. For the inductive case, consider an agent A leaving its initial state. If its transition is an output, then $A = C$ and it determines the branch. Otherwise, the transition is an input and must consume a message produced by another agent. By the induction hypothesis, that agent is already following one of the two branches. Since the branches do not share message types, the input transition of A can only belong to the same branch. Hence agents cannot mix transitions from different branches, and the projected system has the same visible language as T .

Sequential composition. The most interesting case is $F = F_1; F_2$: the language of $\llbracket F \rrbracket$ consists of the concatenations of a word in the language of F_1 with a word in the language of F_2 . By induction, the visible languages of the projections of F_1 and F_2 are subsets of the corresponding languages.

We first build a transition system T that executes the transition system of the projection of F_1 to completion and then executes the transition system of the projection of F_2 . The visible language of T is therefore a subset of the language of $\llbracket F \rrbracket$. Notice, however, that the transition system obtained by projecting F does not necessarily execute the two projections strictly in this order. Since sequential composition is implemented locally, one agent may complete its projection of F_1 and start executing transitions of F_2 while other agents are still executing non visible transitions of F_1 (i.e., inputs).

We show that these additional interleaved executions do not introduce new visible traces. The key argument is a reordering lemma. Consider two consecutive transitions (a) and (b) in an execution of the projection of F , induced respectively by automata transitions $s_1^a \xrightarrow{l^a} s_2^a$ and $s_1^b \xrightarrow{l^b} s_2^b$, where s_1^a belongs to the projection of F_2 and s_1^b belongs to the projection of F_1 . We show that

1. l^a cannot be an output; and
2. the two transitions can be swapped, reaching the same configuration.

For the first point, recall that in the projection of $F_1; F_2$, the first choreography is projected with $RS(F_2)$ as the set of additional agents that must observe its completion. Hence, an agent that can initiate F_2 cannot perform its output before observing the interactions of F_1 on which the beginning of F_2 depends. Therefore, if a transition belonging to F_2 occurs while a transition belonging to F_1 can still occur afterwards, the former transition must be an input and is consequently labelled by τ in the semantics of configurations.

For the second point, let transition (a) be such an input. Since it only updates the local state, environment, and set of received messages of the agent performing it, while leaving the hubs unchanged, it can be commuted with the subsequent transition (b) of F_1 . If (b) belongs to a different agent, the two transitions modify disjoint local states. If they belong to the same agent, their order is already fixed by the local automaton and the situation above cannot arise. Moreover, an input does not remove the event it consumes from the hub, so performing (b) before (a) does not disable (a) . Thus, the two transitions can be swapped and the same configuration is reached.

We can now repeatedly apply this reordering lemma to any execution of the projection of F . Whenever a transition belonging to F_2 occurs before a subsequent transition belonging to F_1 , we swap the two consecutive transitions at the boundary between the two components. Each swap decreases the number of such inversions and does not change the visible trace, since the transition moved to the right is labelled by τ . The process therefore terminates with an execution in which all transitions belonging to the projection of F_1 precede all transitions belonging to the projection of F_2 .

The resulting execution is an execution of T and has the same visible trace as the original one. Hence every visible trace of the projection of $F_1; F_2$ belongs to the visible language of T , which, by the induction hypothesis, is a subset of the language of $\llbracket F_1; F_2 \rrbracket$. $\square$

7. A Case Study

As a running example, consider submitting a travel request at a research institution. The workflow involves the following roles:

- GianLuigi (aka Giangi): the *principal investigator* (PI) of a project;
- one or more *PhD* students;
- the lab *secretary*, who prepares the budget;
- the lab *coordinator*, who approves the budget;
- the institute *director*, who signs the authorisation.

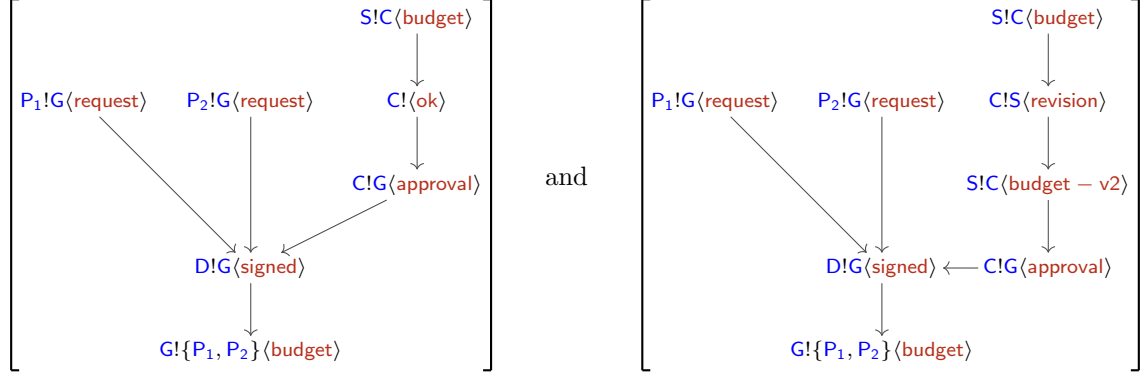
Each student sends the travel request to Giangi. In parallel, the secretary prepares the budget and sends it to the coordinator for approval. The coordinator either returns the budget to the secretary for revision or forwards the approved budget to Giangi. Once the signed authorisation from the director arrives, Giangi sends a copy of the authorised budget to the students.

Steps in this workflow can take place on different hubs: for instance, the exchange of the budget between the secretary and the coordinator, and the authorisation flow between the director and Giangi, need not be visible to the students.

We model the workflow as an F2F-choreography using the following agents: **G** for Giangi, **P₁** and **P₂** for the two students, **S** for the secretary, **C** for the coordinator, and **D** for the director. Interactions are annotated with their intended audience: each message has only one recipient, with the exception of the final budget that is meant for both students, and of the recipient-free *ok* event.

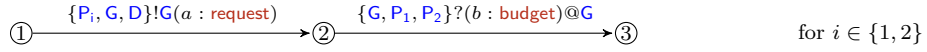
$$\begin{aligned}
F_{\text{init}} &= P_1 \rightsquigarrow G : \text{request} \mid P_2 \rightsquigarrow G : \text{request} \\
F_{\text{budget}} &= S \rightsquigarrow C : \text{budget}; (C \rightsquigarrow \text{ok} + C \rightsquigarrow S : \text{revision}; S \rightsquigarrow C : \text{budget} - v2); C \rightsquigarrow G : \text{approval} \\
F &= (F_{\text{init}} \mid F_{\text{budget}}); D \rightsquigarrow G : \text{signed}; G \rightsquigarrow \{P_1, P_2\} : \text{budget}
\end{aligned}$$

The semantics of the choreography consists of two pomsets, which correspond to the two branches of the choice in F_{budget} :

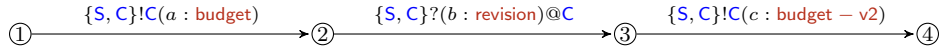


Notice that the director should sign the travel requests and budget only after all of them have been received by Giangi, even if the director does not need to access their content. We now give the projection on each agent.

The projections on the two students are trivial:

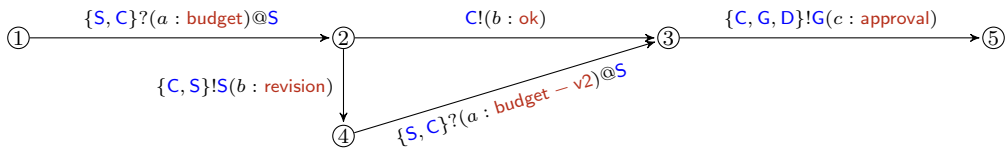


The projection on the secretary S must take into account the possible request for revision:



Notice that the secretary is oblivious of how the workflow continues when the coordinator approves the request without asking for a revision; agent S will wait in state 2 of the automaton above for a revision request. We do not consider this an issue as in this scenario agents are people; should this protocol be enacted by artificial agents then one could let the ok observable to S .

For the coordinator, notice that the approval message must be shared on a hub accessible by D , since they must see this message (and the requests), before signing them and forwarding them to Giangi.



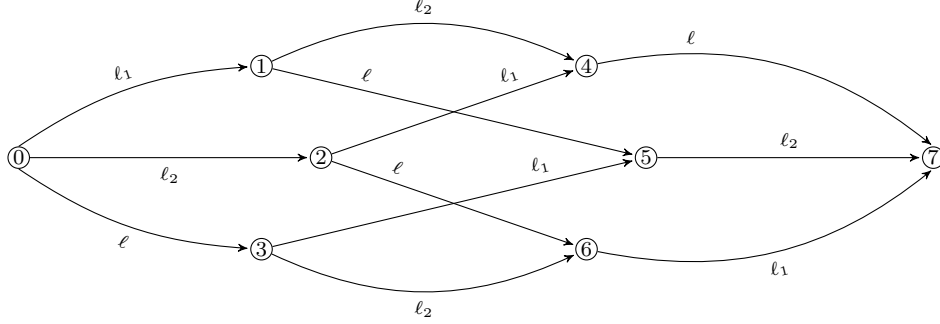
The coordinator receives the budget from the secretary and goes in state 2 to non-deterministically decide if to request a revision or approve the mission straightaway. The latter option is modelled by simply emitting an ok message with no receivers (which our model allows). Such message will be kept in a hub that only the coordinator can access.

Before giving the remaining projections on the director and Giangi, we observe that

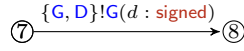
$$(F_{\text{init}} \mid F_{\text{budget}})|_{\mathbf{D}}^{\{\mathbf{D}\}} = (F_{\text{init}} \mid F_{\text{budget}})|_{\mathbf{G}}^{\{\mathbf{D}\}}$$

because none of them is doing output actions in F_{init} or F_{budget} , Giangi is the receiver of the request approval messages only which are also only messages the director must be aware of since it is in the ready set of the continuation of $F_{\text{init}} \mid F_{\text{budget}}$. Therefore, we have that $(F_{\text{init}} \mid F_{\text{budget}})|_{\mathbf{D}}^{\{\mathbf{D}\}} = (F_{\text{init}} \mid F_{\text{budget}})|_{\mathbf{G}}^{\{\mathbf{D}\}}$ is the

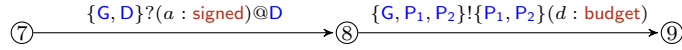
604 following automaton



606 where $\ell = \{\mathbf{C}, \mathbf{G}, \mathbf{D}\}?(c : \text{approval})@\mathbf{C}$, $\ell_1 = \{\mathbf{P}_1, \mathbf{G}, \mathbf{D}\}?(a : \text{request})@\mathbf{P}_1$, and $\ell_2 = \{\mathbf{P}_2, \mathbf{G}, \mathbf{D}\}?(b : \text{request})@\mathbf{P}_2$. To obtain the projection on $\mathbf{D}$ and $\mathbf{G}$ we simply have to add to state 7 the projections of the continuation:



and likewise for the projection on $\mathbf{G}$ iangi:



608 8. An Approach to an Implementation

The realisation of an F2F-choreography, say F , requires the implementation of (i) a mechanism to establish a “master” secret, say M , to let agents initiate sharing secrets to deterministically create the hubs specified in F and (ii) the implementation of the projection operation of Section 4 to obtain the automata corresponding to each agent.⁹

A solution for the first problem exploits similar ideas to the chat protocol of Nearbybytes (cf. <https://github.com/nearbybytes/nearbybytes-chat>). The identity of Nearbybytes agents is determined, in turn, by a public-private key-pair; an agent $\mathbf{A}$ in an F2F-choreography univocally corresponds to a public-private key-pair so that $\mathbf{A}$ can sign messages when authentication or authenticity is required. So the address of a hub shared by any set $\mathbf{A}_1, \dots, \mathbf{A}_n$ of agents in F can be determined by concatenating M with the public keys of each $\mathbf{A}_i$ (in a predetermined order) and deterministically deriving a public-private key-pair from such concatenation. The important point to note is that this address can be independently determined by each agent *locally*, without synchronisation. Indeed, more sophisticated cryptographic strategies are possible (key exchange, etc.) but not necessary to demonstrate full decentralisation of the F2F-choreography model.

Then the question becomes how to implement the automata projected from F . A naive solution is to explicitly represent the automaton so that its transitions can be enacted at runtime.¹⁰ This permits in turn to derive hubs for all communication channels used in a projection. For instance, the address of the hub $\mathfrak{h}$ in a transition

$$s_1 \xrightarrow{\mathfrak{h}! \mathcal{B} \cdot \underline{b} \hookrightarrow (a:t)} s_2$$

626 is the concatenation of the master secret M with the public key of the sender and the public keys of the recipients in $\mathcal{B}$. Each message thus becomes an event in the hub uniquely identified through this mechanism. The message identifier e is simply the hash returned by the `storeEvent` primitive (see Fig. 3).

⁹As said in Section 2.2.3, the secret M can be shared using e.g., social media.

¹⁰This solution could be acceptable if the use of parallel composition is limited in F . In fact, the projection of the parallel composition requires to compute a product of automata (cf. Section 4) which yield a combinatorial explosion of the size of the projection, as also see in Section 7. Optimisations can be applied to tame this problem using for instance on-the-fly techniques instead of generating the automaton upfront.

The causal dependencies b are previous event identifiers (already known, inductively), and are stored using the `blockRefs` field. The message type t is encrypted with a symmetric key also derived from the secret built as above, and stored in a separate block (recall from Section 2 that Nearbybytes also offers a raw block API).

By the architecture of Nearbybytes, no agent will be forced to read messages of the hub unless involved in its coordination mechanism. Indeed, a refinement is possible: “no agent will be *capable of reading* messages of the hub”. This changes the model from “collaborative, but efficiently decentralised” to “secreted”. Several cryptographic mechanisms can easily implement this requirement on top of the scheme described above, and the study of such mechanisms is left for future work.

9. Conclusions, Related, & Future Work

We have introduced F2F-choreographies, an abstract model for the design and the analysis of distributed systems that combine local-first coordination and full decentralisation for confidential interaction. The global views of F2F-choreographies are inspired by global graphs [18, 15] while the local views of F2F-choreographies are a symbolic automata model inspired by Nearbybytes [6]. More precisely, our automata model features group-based interactions and are oblivious to mechanisms to enforce confidentiality; it is the projection operation we defined here that algorithmically determines the *confidentiality infrastructure*, namely the groups of agents that must share cryptographic information in order to execute the choreography on Nearbybytes so to guarantee confidentiality. Following [9], we devise a denotational semantics of F2F-choreographies global views based on pomsets. Partial-order models have long been recognised as a natural semantic foundation for concurrent systems. Pomsets [14] capture causal dependencies while abstracting from unnecessary interleaved executions, and have been successfully employed to model message sequence charts [11] communicating finite-state machines [13], and choreographies [9, 19] (together with models based event structures [20]). Our semantics follows this line of research by interpreting choreographies as sets of pomsets. The main difference is that events denote publication of messages into decentralised append-only hubs.

Choreographies. Choreographies were advocated by the W3C initiative on WSCDL [21] to specify the intended communication behaviour of the whole system from a global point of view while endpoint behaviours are automatically synthesised through projection. In the past two decades choreographies gained momentum both in industry [22] and in academia. Formal models and techniques based on choreographic approaches include behavioural specifications [23], multiparty-session types [24], and choreographic programming [25].

This work departs from common choreographic setting in two fundamental ways. First, communication is modelled as one-to-many dissemination rather than message passing. Second, projection does not only synthesise local behaviours: it also determines the minimal cryptographic infrastructure used by our Nearbybytes engine to enforce confidentiality.

Swarm protocols [2, 3, 4, 5] yield the closest choreographic approach to F2F-choreographies. As F2F-choreographies, swarm protocols feature local-first coordination mechanisms using a decentralised data-space. The main differences are that swarm protocols are conceived to operate in a closed environment, they use broadcast communication, and their logs are totally ordered and designed to ensure eventual consistency (more on this below).

Coordination languages and shared data-spaces. The use of shared communication spaces as coordination mechanisms originates from Linda [26], where processes coordinate indirectly through a shared tuple space. These ideas have inspired a large body of work on coordination languages, including KLAIM [27] and its descendants (cf. [28, 29, 30] and references therein), which extended tuple spaces with notions of mobility, distributed coordination, and locality. Although our hubs resemble tuple spaces in that they provide an indirect communication mechanism, they differ substantially in both semantics and implementation. Hubs are append-only partially ordered logs rather than mutable repositories, and causal dependencies become explicit components of the communication model. Nonetheless there is an interesting overlap between F2F-choreographies and the model proposed in [31] to specify and analyse properties related data flow of

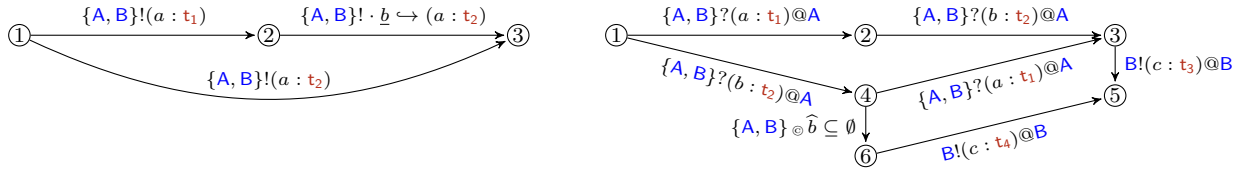
choreographies. Although such model is not featuring local-first coordination, it is partial to autonomy, like F2F-choreographies. Combining the ideas proposed in [31] could be an interesting future development.

Consistency. In this paper we took to an extreme the principles of local-first coordination: we are indeed very partial to autonomy, leaving the problem of guaranteeing consistency of F2F-choreographies in scope for future work. This is indeed an intriguing problem, given the peculiarities of the local-first coordination mechanisms arising from F2F-choreographies. The assumption that the same event cannot happen in two different branches simplifies the problem. In fact, agents can identify branches by inspecting only one event (similarly to e.g., in [2]). This assumption rules out F2F-choreographies such as

$$A \rightsquigarrow t_1; A \rightsquigarrow t_2; B \rightsquigarrow t_3 + A \rightsquigarrow t_2; B \rightsquigarrow t_4 \quad (1)$$

where message t_2 “confuses” B whose output should depend on which of the two branches A takes. In other words, in order to send its output B must wait for a message of type t_2 to appear in the hub. When that happens, t_2 cannot know if such message was produced “on the first branch” (and hence send a message of type t_3) or “on the second branch” (and hence send a message of type t_4). Hence, A and B may end up in inconsistent views of the hub. An F2F setting is not concerned with such problem; as mentioned in Section 2.2, reconciling discordant forks of a decentralised shared state can be dealt with at the application level, and with different strategies.

Nonetheless, consistency is an intriguing problem for F2F-choreographies too. In fact, a generalisation of the projection operation to handle scenarios like the F2F-choreography in (1) could be attained by letting the projected automata check events’ dependencies: m_2 on the two branches differ on the dependency on m_1 . Intuitively, B can distinguish on which branch the message of type t_2 was sent if it depends on a message of type t_1 or not. This can be checked by using query actions in the projection operation (which are also supported in the Nearbybytes implementation). More precisely, we could project A and B as follows:



Namely, B can exploit the pomset structure of the hub using query actions to take its decision provided that A properly declares the causal dependencies of the sent messages. To the best of our knowledge, this mechanism is unique to F2F-choreographies and offers the possibility of exploring sophisticated coordination classes of protocols.

Managing the cryptographic infrastructure. Another interesting direction concerns the generation of secrets to identify hubs. Our construction synthesises a distinct hub for each minimal set of agents that should observe an interaction. One should generate and distribute secrets according to these hubs. An alternative design would derive a cryptographic key for each recipient set and rely on a broadcast encryption scheme [32]. In such a setting, the sender would produce a single ciphertext for an arbitrary subset of agents, while each authorised recipient could decrypt it independently using its own private key, without requiring either one ciphertext per recipient or a dedicated key infrastructure for every subset.

References

- [1] R. Kuhn, W. Reisz, Local-First Cooperation: Autonomy at the Edge, Secured by Crypto, 100% Available, <https://www.infoq.com/articles/local-first-cooperation>, last access on June 14, 2026 (May 2021).
- [2] R. Kuhn, H. Melgratti, E. Tuosto, Behavioural Types for Local-First Software, in: K. Ali, G. Salvaneschi (Eds.), 37th European Conference on Object-Oriented Programming, ECOOP 2023, July 17-21, 2023, Seattle, Washington, United States, Vol. 263 of LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, pp. 15:1–15:28. doi:10.4230/LIPICS.ECOOP.2023.15.

- [3] R. Kuhn, H. C. Melgratti, E. Tuosto, Behavioural Types for Local-First Software (Artifact), Dagstuhl Artifacts Ser. 9 (2) (2023) 14:1–14:5. [doi:10.4230/DARTS.9.2.14](https://doi.org/10.4230/DARTS.9.2.14).
- [4] F. Furbach, L. Clorius, R. Kuhn, H. Melgratti, A. Scalas, E. Tuosto, Compositional Design, Implementation, and Verification of Swarms, in: R. Krebbers, A. Silva (Eds.), 40th European Conference on Object-Oriented Programming, ECOOP 2026, Brussels, Belgium, June 29 - July 3, 2026, Vol. 372 of LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2026, pp. 7:1–7:30. [doi:10.4230/LIPICs.ECOOP.2026.7](https://doi.org/10.4230/LIPICs.ECOOP.2026.7).
- [5] L. Clorius, F. Furbach, R. Kuhn, H. Melgratti, A. Scalas, E. Tuosto, Compositional Design, Implementation, and Verification of Swarms (Artifact), Dagstuhl Artifacts Ser. 12 (1) (2026) 18:1–18:3. [doi:10.4230/DARTS.12.1.18](https://doi.org/10.4230/DARTS.12.1.18).
- [6] The Nearbybytes Group, Nearbybytes: a Local-First Architecture for Post-Cloud Computing, <https://github.com/nearbybytes/community/blob/main/nearbybytes-whitepaper-v0.1.pdf> (2026).
- [7] J. Li, F. Dabek, F2F: reliable storage in open networks, <https://iptps06.cs.ucsb.edu/papers/Li-F2F06.pdf> (2006).
- [8] B. C. Popescu, B. Crispo, A. S. Tanenbaum, Safe and Private Data Sharing with Turtle: Friends Team-Up and Beat the System, in: B. Christianson, B. Crispo, J. A. Malcolm, M. Roe (Eds.), Security Protocols, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 213–220.
- [9] R. Guanciale, E. Tuosto, Realisability of pomsets, Journal of Logic and Algebraic Methods in Programming 108 (2019) 69–89.
- [10] H. Gaifman, V. R. Pratt, Partial order models of concurrency and the computation of functions, in: LICS, 1987, pp. 72–85.
- [11] R. Alur, K. Etessami, M. Yannakakis, Inference of Message Sequence Charts, IEEE Trans. Software Eng. 29 (7) (2003) 623–633.
- [12] J.-P. Katoen, L. Lambert, Pomsets for message sequence charts, Formale Beschreibungstechniken für Verteilte Systeme (1998) 197–208.
- [13] D. Brand, P. Zafropulo, On Communicating Finite-State Machines, JACM 30 (2) (1983) 323–342.
- [14] V. Pratt, Modeling concurrency with partial orders, International Journal of Parallel Programming 15 (1) (1986) 33–71. [doi:10.1007/BF01379149](https://doi.org/10.1007/BF01379149).
- [15] E. Tuosto, R. Guanciale, Semantics of global view of choreographies, Journal of Logic and Algebraic Methods in Programming 95 (2018) 17–40, revised and extended version of [33]. <http://www.cs.le.ac.uk/people/et52/jlamp-with-proofs.pdf>.
- [16] M. Fowler, Event sourcing, <https://martinfowler.com/eaaDev/EventSourcing.html>, accessed: 2026-06-06 (2005).
- [17] Nearbybytes Contributors, Nearbybytes: a data-centric friend-to-friend decentralised storage and communication infrastructure, <https://github.com/nearbybytes/community>, accessed: 2026-06-06 (2026).
- [18] P. Deniérou, N. Yoshida, Multiparty Session Types Meet Communicating Automata, in: ESOP 2012, 2012, pp. 194–213. [doi:10.1007/978-3-642-28869-2_10](https://doi.org/10.1007/978-3-642-28869-2_10).
- [19] L. Edixhoven, S. Jongmans, J. Proença, I. Castellani, Branching pomsets: Design, expressiveness and applications to choreographies, Journal of Logic and Algebraic Methods in Programming 136 (2024) 100919. [doi:10.1016/J.JLAMP.2023.100919](https://doi.org/10.1016/J.JLAMP.2023.100919).
- [20] I. Castellani, M. Dezani-Ciancaglini, P. Giannini, Event structure semantics for multiparty sessions, Journal of Logic and Algebraic Methods in Programming 131 (2023) 100844. [doi:10.1016/J.JLAMP.2022.100844](https://doi.org/10.1016/J.JLAMP.2022.100844).
- [21] N. Kavantzaz, D. Burdett, G. Ritzinger, T. Fletcher, Y. Lafon, Web Services Choreography Description Language Version 1.0., <http://www.w3.org/TR/2004/WD-ws-cdl-10-20041217/>, working Draft 17 December 2004 (2004).
- [22] J. Bonér, Reactive Microsystems - The Evolution Of Microservices At Scale, O'Reilly, 2018.
- [23] H. Hüttel, I. Lanese, V. T. Vasconcelos, L. Caires, M. Carbone, P. Deniérou, D. Mostrous, L. Padovani, A. Ravara, E. Tuosto, H. T. Vieira, G. Zavattaro, Foundations of Session Types and Behavioural Contracts, ACM Comput. Surv. 49 (1) (2016) 3:1–3:36.
- [24] K. Honda, N. Yoshida, M. Carbone, Multiparty asynchronous session types, JACM 63 (1) (2016) 9:1–9:67, extended version of [34]. [doi:10.1145/2827695](https://doi.org/10.1145/2827695).
- [25] F. Montesi, Introduction to Choreographies, Cambridge University Press, 2023.
- [26] N. Carriero, D. Gelernter, Linda in context, CACM 32 (4) (1989) 444–458.
- [27] L. Bettini, V. Bono, R. De Nicola, G. Ferrari, D. Gorla, M. Loreti, E. Moggi, R. Pugliese, E. Tuosto, B. Venneri, The Klaim Project: Theory and Practice, in: C. Priami (Ed.), Global Computing. Programming Environments, Languages, Security, and Analysis of Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 88–150.
- [28] M. Boreale, F. Corradini, M. Loreti, R. Pugliese (Eds.), Models, Languages, and Tools for Concurrent and Distributed Programming - Essays Dedicated to Rocco De Nicola on the Occasion of His 65th Birthday, Vol. 11665 of Lecture Notes in Computer Science, Springer, 2019. [doi:10.1007/978-3-030-21485-2](https://doi.org/10.1007/978-3-030-21485-2).
- [29] R. De Nicola, G. Ferrari, R. Pugliese, F. Tiezzi, A formal approach to the engineering of domain-specific distributed systems, Journal of Logic and Algebraic Methods in Programming 111 (2020) 100511, extended version of [35]. [doi:https://doi.org/10.1016/j.jlamp.2019.100511](https://doi.org/10.1016/j.jlamp.2019.100511).
- [30] L. Bettini, G. Ferrari, M. Loreti, R. Pugliese, F. Tiezzi, E. Tuosto, Klaim in the Making, in: T. Margaria, B. Steffen (Eds.), Leveraging Applications of Formal Methods, Verification and Validation. REoCAS Colloquium in Honor of Rocco De Nicola - 12th International Symposium, ISoLA 2024, Crete, Greece, October 27-31, 2024, Proceedings, Part I, Vol. 15219 of Lecture Notes in Computer Science, Springer, 2024, pp. 27–49. [doi:10.1007/978-3-031-73709-1_3](https://doi.org/10.1007/978-3-031-73709-1_3).
- [31] R. Bruni, A. Corradini, F. Gadducci, H. C. Melgratti, U. Montanari, E. Tuosto, Data-Driven Choreographies à la Klaim, in: Boreale et al. [28], pp. 170–190. [doi:10.1007/978-3-030-21485-2](https://doi.org/10.1007/978-3-030-21485-2).
- [32] D. Boneh, C. Gentry, B. Waters, Collusion resistant broadcast encryption with short ciphertexts and private keys, in: Annual international cryptology conference, Springer, 2005, pp. 258–275.

- [33] R. Guanciale, E. Tuosto, An abstract semantics of the global view of choreographies, in: Proceedings 9th Interaction and Concurrency Experience, ICE 2016, Heraklion, Greece, 8-9 June 2016., 2016, pp. 67–82.
- [34] K. Honda, N. Yoshida, M. Carbone, Multiparty asynchronous session types, in: POPL, POPL '08, Association for Computing Machinery, New York, NY, USA, 2008, p. 273–284. [doi:10.1145/1328438.1328472](https://doi.org/10.1145/1328438.1328472).
- [35] R. De Nicola, G. Ferrari, R. Pugliese, F. Tiezzi, A Formal Approach to the Engineering of Domain-Specific Distributed Systems, in: G. Di Marzo Serugendo, M. Loreti (Eds.), Coordination Models and Languages, Springer, 2018, pp. 110–141.